Pydantic: Pythonにおけるデータ検証の簡素化

情報源: Real Python | **著者:** Harrison Hoffman | **公開日:** 2024-04-10

Pydanticは、コードベースの堅牢性と信頼性を高めるために設計された、Python向けの強力なデータ検証および設定管理ライブラリです。変数が整数であるかどうかをチェックするような基本的なタスクから、高度に入れ子になった辞書のキーや値が正しいデータ型を持っているかを確認するような複雑なタスクまで、Pydanticは最小限のボイラープレートコードであらゆるデータ検証シナリオに対応できます。

このチュートリアルで学ぶこと：

- PydanticのBaseModelを使った**データスキーマ**の操作
- 複雑なユースケース向けの**カスタムバリデータの作成**
- Pydanticの@validate_callによる**関数引数の検証**
- pydantic-settingsを使った設定管理と**アプリケーションの設定**

このチュートリアルを通じて、Pydanticの機能に関する実践的な例を学びます。終える頃には、独自の検証ユースケースに対応するための確かな基礎が身につくでしょう。このチュートリアルを始める前に、Pythonおよびオブジェクト指向プログラミングに関する中級程度の理解があると役立ちます。

1 PythonのPydanticライブラリ

Pythonの大きな魅力の1つは、動的型付け言語であることです。動的型付けとは、コンパイル時に明示的に型を宣言する静的型付け言語とは異なり、実行時に変数の型が決定されることを意味します。動的型付けは迅速な開発や使いやすさには非常に適していますが、実際のアプリケーションでは、より堅牢な型チェックやデータ検証が必要になることがよくあります。そこで役立つのが、PythonのPydanticライブラリです。

Pydanticは急速に人気を集め、現在ではPythonで最も広く使用されているデータ検証ライブラリとなっています。この最初のセクションでは、Pydanticの概要と、このライブラリの強力な機能のプレビューを紹介します。また、Pydanticのインストール方法と、このチュートリアルで必要となる追加の依存関係についても学びます。

1.1 Pydanticに親しむ

Pydanticは、型ヒントを活用してデータスキーマの検証やシリアライズを容易にする強力なPythonライブラリです。これにより、コードはより堅牢で、読みやすく、簡潔になり、デバッグも容易になります。また、Pydanticは多くの一般的な静的型付けツールやIDEと適切に連携するため、コードを実行する前にスキーマの問題を捉えることができます。

Pydanticの主な特徴は以下の通りです：

- **カスタマイズ性**：Pydanticで検証できるデータの種類に制限はほぼありません。Pythonの基本型から高度に入れ子になったデータ構造まで、Pydanticを使えばほぼすべてのPythonオブジェクトを検証し、シリアライズできます。

- **柔軟性**：Pydanticでは、データの検証をどれくらい厳格に行うか、あるいは寛容に行うかを制御できます。場合によっては、受け取ったデータを適切な型に強制変換（キャスト）したいこともあるでしょう。例えば、本来は浮動小数点数（float）であるべきデータを整数（int）として受け取った場合に許容するなどです。一方で、受け取るデータ型を厳格に強制したい場合もあります。Pydanticはそのどちらにも対応可能です。
- **シリアライズ**：Pydanticオブジェクトは辞書型やJSON文字列としてシリアライズおよびデシリアライズできます。これは、PydanticオブジェクトとJSONをシームレスに相互変換できることを意味します。この機能により、自己文書化するAPIの構築や、JSONスキーマをサポートするほぼあらゆるツールとの連携が可能になります。
- **パフォーマンス**：コアとなる検証ロジックがRustで書かれているため、Pydanticは非常に高速です。このパフォーマンス上の強みにより、特に大量のリクエストに対応する必要があるREST APIのような高スループットなアプリケーションにおいて、迅速かつ信頼性の高いデータ処理が可能になります。
- **エコシステムと業界での採用状況**：Pydanticは、FastAPI、LangChain、Polarsなど、数多くの人気Pythonライブラリの依存関係となっています。また、大手IT企業の大半や、他の多くの業界で広く使用されています。これは、Pydanticのコミュニティによるサポート、信頼性、および堅牢性の証です。

これらは、Pydanticを魅力的なデータ検証ライブラリにしている主要な機能のほんの一部であり、このチュートリアルを通じて実際の動作を確認していきます。次は、Pydanticと各種依存関係のインストール方法の概要について説明します。

1.2 Pydanticのインストール

PydanticはPyPIで公開されており、pipを使ってインストールできます。ターミナルまたはコマンドプロンプトを開き、新しい仮想環境を作成してから、次のコマンドを実行してPydanticをインストールします。

```
(venv) $ python -m pip install pydantic
```

このコマンドにより、PyPIからPydanticの最新バージョンがマシンにインストールされます。インストールが成功したことを確認するには、Python REPLを起動してPydanticをインポートします。

```
>>> import pydantic
```

エラーなしでインポートが実行できれば、Pydanticのインストールは成功であり、システムにPydanticのコア機能がインストールされたことになります。

1.3 オプションの依存関係の追加

Pydanticでは、オプションの依存関係もあわせてインストールできます。たとえば、このチュートリアルではメールアドレスの検証を扱いますが、以下のようにして関連する依存関係をインストールに含めることができます。

```
(venv) $ python -m pip install "pydantic[email]"
```

Pydanticには設定管理用の個別のパッケージもあり、これも本チュートリアルで取り上げます。これをインストールするには、次のコマンドを実行します。

```
(venv) $ python -m pip install pydantic-settings
```

これで、このチュートリアルで必要なすべての依存関係がインストールされ、Pydanticの学習を始める準備が整いました。まずは、Pydanticにおけるデータスキーマの主要な定義方法である「モデル」から説明していきます。

2 モデルの使用

Pydanticでデータスキーマを定義する主な方法は、モデルを使用することです。Pydanticモデルは、Pythonのdataclassに似たオブジェクトであり、型注釈（アノテーション）付きのフィールドを使用してエンティティに関するデータを定義および格納します。dataclassとは異なり、Pydanticの主な焦点は自動データパース（解析）、検証（バリデーション）、およびシリアライズ（直列化）にあります。

これを理解する最良の方法は、独自のモデルを作成することです。次にそれを行っていきましょう。

2.1 PydanticのBaseModelの操作

人事部門が従業員情報を管理するために使用するアプリケーションを構築しており、新しい従業員情報が正しい形式であることを検証する方法が必要であると仮定しましょう。例えば、各従業員にはID、名前、メールアドレス、生年月日、給与、部署、福利厚生の選択情報が含まれている必要があります。これはPydanticモデルの格好のユースケースです！

従業員モデルを定義するには、Pydanticの `BaseModel` を継承するクラスを作成します。

pydantic_models.py

```
from datetime import date
from uuid import UUID, uuid4
from enum import Enum
from pydantic import BaseModel, EmailStr

class Department(Enum):
    HR = "HR"
    SALES = "SALES"
    IT = "IT"
    ENGINEERING = "ENGINEERING"

class Employee(BaseModel):
    employee_id: UUID = uuid4()
    name: str
    email: EmailStr
    date_of_birth: date
    salary: float
    department: Department
    elected_benefits: bool
```

まず、従業員モデルの定義に必要な依存関係をインポートします。次に、社内のさまざまな部署を表す列挙型（enum）を作成し、これを使用して従業員モデルの `department` フィールドをアノテーションします。

次に、`BaseModel` を継承するPydanticモデル `Employee` を定義し、アノテーションを通じて従業員フィールドの名前と期待される型を定義します。以下は、`Employee` で定義した各フィールドの内訳と、`Employee` オブジェクトがインスタンス化される際にPydanticがそれをどのように検証するかの説明です。

- `employee_id`: 情報を保存したい従業員のUUIDです。`UUID` アノテーションを使用することで、Pydanticはこのフィールドが常に有効なUUIDであることを保証しま

す。`uuid4()` の呼び出しで指定したように、デフォルトで `Employee` の各インスタンスにUUIDが割り当てられます。

- `name`: 従業員の名前です。Pydanticはこれが文字列（`str`）であることを想定します。
- `email`: Pydanticは内部でPythonの `email-validator` ライブラリを使用して、各従業員の `email` が有効であることを保証します。
- `date_of_birth`: 各従業員の生年月日は、Pythonの `datetime` モジュールの `date` でアノテーションされているとおり、有効な日付である必要があります。`date_of_birth` に文字列を渡すと、Pydanticはそれを解析して `date` オブジェクトに変換しようとします。
- `salary`: 従業員の給与であり、浮動小数点数（`float`）であることが想定されます。
- `department`: 各従業員の部署は、`Department` enumで定義されている `HR`、`SALES`、`IT`、`ENGINEERING` のいずれかである必要があります。
- `elected_benefits`: このフィールドは従業員が福利厚生を選択したかどうかを保持し、Pydanticはこれが真偽値（`bool`）であることを想定します。

`Employee` オブジェクトを作成する最も簡単な方法は、他のPythonオブジェクトと同様にインスタンス化することです。これを行うには、PythonのREPLを開き、次のコードを実行します。

```
>>> from pydantic_models import Employee

>>> Employee(
...     name="Chris DeTuma",
...     email="cdetuma@example.com",
...     date_of_birth="1998-04-02",
...     salary=123_000.00,
...     department="IT",
...     elected_benefits=True,
... )
Employee(
    employee_id=UUID('73636d47-373b-40cd-a005-4819a84d9ea7'),
    name='Chris DeTuma',
    email='cdetuma@example.com',
    date_of_birth=datetime.date(1998, 4, 2),
    salary=123000.0,
    department=<Department.IT: 'IT'>,
    elected_benefits=True
)
```

このブロックでは、`Employee` をインポートし、必要な従業員フィールドをすべて指定してオブジェクトを作成します。Pydanticは渡されたフィールドを正常に検証および型変換（coercion）し、有効な `Employee` オブジェクトを作成します。Pydanticが日付文字列を `date` オブジェクトに、`IT` 文字列を対応する `Department` enumに自動的に変換していることに注目してください。

次に、無効なデータを `Employee` インスタンスに渡そうとした場合にPydanticがどのように応答するかを見てみましょう。

```
>>> Employee(
...     employee_id="123",
...     name=False,
...     email="cdetumaexamplecom",
...     date_of_birth="1939804-02",
```

```
...     salary="high paying",
...     department="PRODUCT",
...     elected_benefits=300,
... )

Traceback (most recent call last):
pydantic_core._pydantic_core.ValidationError: 7 validation errors for
Employee

employee_id
  Input should be a valid UUID, invalid length: expected length 32 for
  simple format, found 3 [type=uuid_parsing, input_value='123',
  input_type=str] For further information visit
  https://errors.pydantic.dev/2.6/v/uuid_parsing

name
  Input should be a valid string [type=string_type, input_value=False,
  input_type=bool] For further information visit
  https://errors.pydantic.dev/2.6/v/string_type

email
  value is not a valid email address: The email address is not valid.
  It must have exactly one @-sign. [type=value_error,
  input_value='cdetumaexamplecom', input_type=str]

date_of_birth
  Input should be a valid date or datetime, invalid date separator,
  expected `-` [type=date_from_datetime_parsing,
  input_value='1939804-02', input_type=str] For further information
  visit https://errors.pydantic.dev/2.6/v/date_from_datetime_parsing

salary
  Input should be a valid number, unable to parse string as a number
  [type=float_parsing, input_value='high paying', input_type=str]
  For further information visit
  https://errors.pydantic.dev/2.6/v/float_parsing

department
  Input should be 'HR', 'SALES', 'IT' or 'ENGINEERING'
  [type=enum, input_value='PRODUCT', input_type=str]

elected_benefits
  Input should be a valid boolean, unable to interpret input
  [type=bool_parsing, input_value=300, input_type=int]
  For further information visit
  https://errors.pydantic.dev/2.6/v/bool_parsing
```

この例では、無効なデータフィールドを持つ Employee オブジェクトを作成しました。Pydanticは各フィールドに対して詳細なエラーメッセージを出力し、何が期待されていたか、何が受け取られたか、そしてエラーの詳細についてどこで学べるかを教えてくれます。

これをフィールドごとに検証するには、以下のウィジェットを使用します。このウィジェットは、Pydanticが date_of_birth を date に、department を Department のメンバーに型変換しつつ、他のフィールドについては渡された値をそのまま保持するという、記事の有効な例からスタートします。任意のフィールドに対して拒否される値（例：date_of_birth に対する '1939804-02' など）を選択すると、フッターが作成された Employee からバリデー

ションエラーへと切り替わるのを確認できます。



この詳細なバリデーションは、無効なデータが `Employee` に保存されるのを防ぐため非常に強力です。また、エラーなくインスタンス化された `Employee` オブジェクトには期待通りのデータが含まれているという確信が得られ、コードの後続処理や他のアプリケーションでもこのデータを信頼して利用することができます。

Pydanticの `BaseModel` には、辞書（dict）やJSONなどの他のオブジェクトからモデルを簡単に作成できるようにする一連のメソッドが用意されています。たとえば、辞書から `Employee` オブジェクトをインスタンス化したい場合は、`.model_validate()` クラスメソッドを使用できます。

```
>>> new_employee_dict = {
...     "name": "Chris DeTuma",
...     "email": "cdetuma@example.com",
...     "date_of_birth": "1998-04-02",
...     "salary": 123_000.00,
...     "department": "IT",
...     "elected_benefits": True,
... }

>>> Employee.model_validate(new_employee_dict)
Employee(
    employee_id=UUID('73636d47-373b-40cd-a005-4819a84d9ea7'),
    name='Chris DeTuma',
    email='cdetuma@example.com',
    date_of_birth=datetime.date(1998, 4, 2),
    salary=123000.0,
    department=<Department.IT: 'IT'>,
    elected_benefits=True
)
```

ここでは、従業員フィールドを持つ辞書 `new_employee_dict` を作成し、それを `.model_validate()` に渡して `Employee` インスタンスを作成します。内部では、Pydanticが辞書の各エントリを検証し、期待するデータに準拠しているか確認します。データが無効な

場合、Pydanticは先ほど見たのと同様にエラーを発生させます。また、辞書に必要なフィールドが欠けている場合も通知されます。

.model_validate_json() を使用すると、JSONオブジェクトに対しても同じことができます。

```
>>> new_employee_json = """
...  {"employee_id":"d2e7b773-926b-49df-939a-5e98cbb9c9eb",
...  "name":"Eric Slogrenta",
...  "email":"eslogrenta@example.com",
...  "date_of_birth":"1990-01-02",
...  "salary":125000.0,
...  "department":"HR",
...  "elected_benefits":false}
...  """

>>> new_employee = Employee.model_validate_json(new_employee_json)
>>> new_employee
Employee(
    employee_id=UUID('d2e7b773-926b-49df-939a-5e98cbb9c9eb'),
    name='Eric Slogrenta',
    email='eslogrenta@example.com',
    date_of_birth=datetime.date(1990, 1, 2),
    salary=125000.0,
    department=<Department.HR: 'HR'>,
    elected_benefits=False
)
```

この例では、new_employee_json は従業員フィールドを格納する有効なJSON文字列であり、.model_validate_json() を使用して new_employee_json から Employee オブジェクトを検証して作成します。地味に思えるかもしれませんが、JSONからPydanticモデルを作成して検証する機能は非常に強力です。なぜなら、JSONはWeb上でデータを転送する最も人気のある方法の1つだからです。これが FastAPI がREST APIを作成する際にPydanticに依存している理由の1つです。

Pydanticモデルを辞書やJSONとしてシリアライズすることもできます。

```
>>> new_employee.model_dump()
{
    'employee_id': UUID('d2e7b773-926b-49df-939a-5e98cbb9c9eb'),
    'name': 'Eric Slogrenta',
    'email': 'eslogrenta@example.com',
    'date_of_birth': datetime.date(1990, 1, 2),
    'salary': 125000.0,
    'department': <Department.HR: 'HR'>,
    'elected_benefits': False
}

>>> new_employee.model_dump_json()
'{"employee_id":"d2e7b773-926b-49df-939a-5e98cbb9c9eb","name":"Eric
   Slogrenta","email":"eslogrenta@example.com","date_of_birth
   ":"1990-01-02","salary":125000.0,"department":"HR","elected_benefits
   ":false}'
```

ここでは、.model_dump() と .model_dump_json() を使用して、new_employee モデルをそれぞれ辞書とJSON文字列に変換します。.model_dump_json() が date_of_birth と department を文字列として格納したJSONオブジェクトを返すことに注目してください。

Pydanticはすでにこれらのフィールドを検証してモデルをJSONに変換していますが、後続の処理でこのJSONを使用する側は、`date_of_birth` が有効な `date` である必要があり、`department` が `Department` enumのカテゴリである必要があることを知りません。これを解決するために、`Employee` モデルから JSON Schema（JSON スキーマ） を作成できます。

JSON Schemaは、JSONオブジェクトでどのようなフィールドが期待され、どのような値が表現されるかを説明します。これは `Employee` クラス定義のJSON版と考えることができます。以下は、`Employee` のJSON Schemaを生成する方法です。

```
>>> Employee.model_json_schema()
{
    '$defs': {
        'Department': {
            'enum': ['HR', 'SALES', 'IT', 'ENGINEERING'],
            'title': 'Department',
            'type': 'string'
        }
    },
    'properties': {
        'employee_id': {
            'default': '73636d47-373b-40cd-a005-4819a84d9ea7',
            'format': 'uuid',
            'title': 'Employee Id',
            'type': 'string'
        },
        'name': {'title': 'Name', 'type': 'string'},
        'email': {
            'format': 'email',
            'title': 'Email',
            'type': 'string'
        },
        'date_of_birth': {
            'format': 'date',
            'title': 'Date Of Birth',
            'type': 'string'
        },
        'salary': {'title': 'Salary', 'type': 'number'},
        'department': {'$ref': '#/$defs/Department'},
        'elected_benefits': {'title': 'Elected Benefits', 'type': '
   boolean'}
    },
    'required': [
        'name',
        'email',
        'date_of_birth',
        'salary',
        'department',
        'elected_benefits'
    ],
    'title': 'Employee',
    'type': 'object'
}
```

`.model_json_schema()` を呼び出すと、モデルのJSON Schemaを表す辞書が得られます。最初のエントリには、`department` が取り得る値が表示されています。また、フィールドがどのようにフォーマットされるべきかに関する情報も表示されます。例えば、このJSON

Schemaによると、employee_id はUUID、date_of_birth は日付であることが期待されています。

json.dumps() を使用してJSON SchemaをJSON文字列に変換できます。これにより、ほぼあらゆるプログラミング言語で Employee モデルから生成されたJSONオブジェクトを検証できるようになります。言い換えれば、Pydanticは入力データを検証してJSONとしてシリアライズできるだけでなく、JSON Schemaを通じて他のプログラミング言語がモデルのデータを検証するために必要な情報も提供します。

これで、データを検証およびシリアライズするためにPydanticの BaseModel を使用する方法が理解できました。次は、フィールドを使用してバリデーションをさらにカスタマイズする方法について学びます。

2.2 フィールド（Field）を使用したカスタマイズとメタデータ

ここまでで、Employee モデルは各フィールドのデータ型を検証し、email、date_of_birth、department などの一部のフィールドが有効な形式であることを保証しています。しかし、さらに salary が正の数であること、name が空文字列でないこと、email に自社のドメイン名が含まれていることを保証したいとしましょう。これを実現するために、Pydanticの Field クラスを使用できます。

Field クラスを使用すると、モデルのフィールドをカスタマイズし、メタデータを追加できます。これがどのように動作するか、次の例を見てみましょう。

pydantic_models.py

```
from datetime import date
from uuid import UUID, uuid4
from enum import Enum
from pydantic import BaseModel, EmailStr, Field

class Department(Enum):
    HR = "HR"
    SALES = "SALES"
    IT = "IT"
    ENGINEERING = "ENGINEERING"

class Employee(BaseModel):
    employee_id: UUID = Field(default_factory=uuid4, frozen=True)
    name: str = Field(min_length=1, frozen=True)
    email: EmailStr = Field(pattern=r".+@example\.com$")
    date_of_birth: date = Field(alias="birth_date", repr=False, frozen=
   True)
    salary: float = Field(alias="compensation", gt=0, repr=False)
    department: Department
    elected_benefits: bool
```

ここでは、以前使用した他の依存関係とともに Field をインポートし、Employee のフィールドの一部にデフォルト値を割り当てています。以下は、フィールドにバリデーションとメタデータを追加するために使用した Field パラメータの内訳です。

- default_factory: デフォルト値を生成する呼び出し可能オブジェクト（callable）を定義するために使用します。上の例では、default_factory に uuid4 を設定しています。これにより、必要なときに uuid4() が呼び出され、employee_id のランダムなUUIDが生成されます。より柔軟に対応するために 無名関数（lambda） を使用することもできます。

- **frozen**: フィールドを不変（イミュータブル）にするために設定できる真偽値（Boolean）パラメータです。つまり、frozen が True に設定されている場合、モデルがインスタンス化された後に対応するフィールドを変更することはできません。この例では、frozen パラメータを使用して employee_id、name、および date_of_birth を不変にしています。
- **min_length**: min_length と max_length を使用して、文字列フィールドの長さを制御できます。上の例では、name が少なくとも1文字以上であることを保証しています。
- **pattern**: 文字列フィールドの場合、そのフィールドに期待するパターンに一致する 正規表現 を pattern に設定できます。たとえば、email に対して上の例の正規表現を使用すると、Pydanticはすべてのメールアドレスが @example.com で終わることを保証します。
- **alias**: フィールドにエイリアス（別名）を割り当てたい場合にこのパラメータを使用します。たとえば、date_of_birth を birth_date と呼んだり、salary を compensation と呼んだりすることを許可できます。モデルをインスタンス化またはシリアライズする際にこれらのエイリアスを使用できます。
- **gt**: 「greater than（～より大きい）」の略であるこのパラメータは、数値フィールドの最小値を設定するために使用されます。この例では、gt=0 を設定することで salary が常に正の数であることを保証しています。Pydanticには、「less than（～より小さい）」の略である lt など、他の 数値制約 も用意されています。
- **repr**: この真偽値パラメータは、モデルの文字列表現にフィールドが表示されるかどうかを決定します。この例では、Employee インスタンスを出力したときに date_of_birth や salary は表示されません。

この追加のバリデーションの動きを確認するために、不適切なデータで Employee モデルを作成しようとしたときに何が起こるか注目してください。

```
>>> from pydantic_models import Employee

>>> incorrect_employee_data = {
...     "name": "",
...     "email": "cdetuma@fakedomain.com",
...     "birth_date": "1998-04-02",
...     "salary": -10,
...     "department": "IT",
...     "elected_benefits": True,
... }

>>> Employee.model_validate(incorrect_employee_data)
Traceback (most recent call last):

pydantic_core._pydantic_core.ValidationError: 3 validation errors for
Employee
name
  String should have at least 1 character [type=string_too_short,
   input_value='', input_type=str] For further information visit
    https://errors.pydantic.dev/2.6/v/string_too_short
email
  String should match pattern '.+@example\.com$'
  [type=string_pattern_mismatch,
  input_value='cdetuma@fakedomain.com', input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/string_pattern_mismatch
salary
```

```
  Input should be greater than 0 [type=greater_than, input_value=-10,
  input_type=int] For further information visit
  https://errors.pydantic.dev/2.6/v/greater_than
```

ここでは、更新された Employee モデルをインポートし、不適切なデータを含む辞書を検証しようとしています。これに対し、Pydanticは name が少なくとも1文字以上である必要があること、email が自社のドメイン名と一致する必要があること、salary が0より大きい必要があることを示す3つのバリデーションエラーを返します。

それでは、正しい Employee データを検証したときに得られる追加機能に注目してください。

```
>>> employee_data = {
...     "name": "Clyde Harwell",
...     "email": "charwell@example.com",
...     "birth_date": "2000-06-12",
...     "compensation": 100_000,
...     "department": "ENGINEERING",
...     "elected_benefits": True,
... }

>>> employee = Employee.model_validate(employee_data)
>>> employee
Employee(
    employee_id=UUID('614c6f75-8528-4272-9cfc-365ddfafebd9'),
    name='Clyde Harwell',
    email='charwell@example.com',
    department=<Department.ENGINEERING: 'ENGINEERING'>,
    elected_benefits=True)

>>> employee.salary
100000.0

>>> employee.date_of_birth
datetime.date(2000, 6, 12)
```

このブロックでは、辞書を作成し、.model_validate() を使用して Employee モデルを作成します。employee_data で、date_of_birth の代わりに birth_date を、salary の代わりに compensation を使用していることに注目してください。Pydanticはこれらのエイリアスを認識し、内部でそれらの値を正しいフィールド名に割り当てます。

repr=False を設定したため、Employee の文字列表現に salary と date_of_birth が表示されていないことがわかります。それらの値を確認するには、属性として明示的にアクセスする必要があります。最後に、凍結（frozen）されたフィールドを変更しようとしたときに何が起こるか注目してください。

```
>>> employee.department = "HR"
>>> employee.name = "Andrew TuGrendele"
Traceback (most recent call last):

pydantic_core._pydantic_core.ValidationError: 1
validation error for Employee
name
  Field is frozen [type=frozen_field, input_value='Andrew TuGrendele',
  input_type=str]
  For further information visit
  https://errors.pydantic.dev/2.6/v/frozen_field
```

ここではまず、department の値を IT から HR に変更します。department は凍結されたフィールドではないため、これは全く問題ありません。しかし、name を変更しようとすると、Pydanticは name が凍結フィールドであることを示すエラーを出力します。

これで、Pydanticの BaseModel と Field クラスについてしっかりと把握できました。これらだけでも、データスキーマにさまざまな検証ルールやメタデータを定義できますが、時にはこれだけでは十分でないこともあります。次は、Pydanticのバリデータ（validator）を使用して、フィールドバリデーションをさらに一歩進める方法を学びます。

3　バリデータでの作業

これまでは、Pydanticの BaseModel を使用して事前定義された型でモデルフィールドを検証し、Field を組み込んでバリデーションをさらにカスタマイズしてきました。BaseModel と Field だけでもある程度のことは可能ですが、カスタムロジックを必要とするより複雑なバリデーションシナリオでは、Pydanticのバリデータを使用する必要があります。

バリデータを使用すると、関数で表現できるほぼあらゆるバリデーションロジックを実行できます。次にその方法を見ていきましょう。

3.1　モデルとフィールドの検証

従業員の例を続けて、あなたの会社に「少なくとも18歳以上の従業員のみを雇用する」というポリシーがあると仮定します。新しい Employee オブジェクトを作成するときは常に、従業員が18歳より年上であることを確認する必要があります。これを処理するために、age フィールドを追加して Field クラスを使用し、従業員が18歳以上であることを強制することもできます。しかし、すでに従業員の生年月日を保存しているため、これは冗長に思えます。

より良い解決策は、Pydanticのフィールドバリデータを使用することです。フィールドバリデータを使用すると、モデルにクラスメソッドを追加することで、BaseModel フィールドにカスタムバリデーションロジックを適用できます。すべての従業員が18歳以上であることを強制するには、Employee モデルに次のフィールドバリデータに追加します。

pydantic_models.py

```
from datetime import date
from uuid import UUID, uuid4
from enum import Enum
from pydantic import BaseModel, EmailStr, Field, field_validator

class Department(Enum):
    HR = "HR"
    SALES = "SALES"
    IT = "IT"
    ENGINEERING = "ENGINEERING"

class Employee(BaseModel):
    employee_id: UUID = Field(default_factory=uuid4, frozen=True)
    name: str = Field(min_length=1, frozen=True)
    email: EmailStr = Field(pattern=r".+@example\.com$")
    date_of_birth: date = Field(alias="birth_date", repr=False, frozen=
   True)
    salary: float = Field(alias="compensation", gt=0, repr=False)
    department: Department
    elected_benefits: bool
```

```
    @field_validator("date_of_birth")
    @classmethod
    def check_valid_age(cls, date_of_birth: date) -> date:
        today = date.today()
        eighteen_years_ago = date(today.year - 18, today.month, today.
   day)

        if date_of_birth > eighteen_years_ago:
            raise ValueError("Employees must be at least 18 years old."
   )

        return date_of_birth
```

このブロックでは、field_validator をインポートし、それを使って Employee 内の .check_valid_age() というクラスメソッドをデコレートしています。フィールドバリデータはクラスメソッドとして定義する必要があります。.check_valid_age() では、今日から18年前の日付を計算しています。従業員の date_of_birth がその日付より後の場合、エラーが発生します。

このバリデータがどのように機能するか確認するために、次の例を見てみましょう。

```
>>> from pydantic_models import Employee
>>> from datetime import date, timedelta

>>> young_employee_data = {
...     "name": "Jake Bar",
...     "email": "jbar@example.com",
...     "birth_date": date.today() - timedelta(days=365 * 17),
...     "compensation": 90_000,
...     "department": "SALES",
...     "elected_benefits": True,
... }

>>> Employee.model_validate(young_employee_data)
Traceback (most recent call last):

pydantic_core._pydantic_core.ValidationError:
1 validation error for Employee
birth_date
  Value error, Employees must be at least 18 years old.
  [type=value_error, input_value=datetime.date(2007, 4, 10),
  input_type=date]
  For further information visit
  https://errors.pydantic.dev/2.6/v/value_error
```

この例では、現在の日付から17年前の birth_date を指定しています。young_employee_data を検証するために .model_validate() を呼び出すと、従業員は少なくとも18歳でなければならないというエラーが発生します。

想像通り、Pydanticの field_validator() を使うとフィールドの検証を自由にカスタマイズできます。ただし、複数のフィールドを相互に比較したり、モデル全体を検証したりしたい場合には field_validator() は機能しません。このような場合には、モデルバリデータを使用する必要があります。

例として、あなたの会社がIT部門でのみ契約社員を雇用していると仮定しましょう。このため、IT部門の従業員は福利厚生の対象外となり、elected_benefits フィールドは False で

ある必要があります。Pydanticの model_validator() を使用して、この制約を強制できます。

pydantic_models.py

```
from typing import Self
from datetime import date
from uuid import UUID, uuid4
from enum import Enum
from pydantic (
    BaseModel,
    EmailStr,
    Field,
    field_validator,
    model_validator,
)

class Department(Enum):
    HR = "HR"
    SALES = "SALES"
    IT = "IT"
    ENGINEERING = "ENGINEERING"

class Employee(BaseModel):
    employee_id: UUID = Field(default_factory=uuid4, frozen=True)
    name: str = Field(min_length=1, frozen=True)
    email: EmailStr = Field(pattern=r".+@example\.com$")
    date_of_birth: date = Field(alias="birth_date", repr=False, frozen=
   True)
    salary: float = Field(alias="compensation", gt=0, repr=False)
    department: Department
    elected_benefits: bool

    @field_validator("date_of_birth")
    @classmethod
    def check_valid_age(cls, date_of_birth: date) -> date:
        today = date.today()
        eighteen_years_ago = date(today.year - 18, today.month, today.
   day)

        if date_of_birth > eighteen_years_ago:
            raise ValueError("Employees must be at least 18 years old."
   )

        return date_of_birth

    @model_validator(mode="after")
    def check_it_benefits(self) -> Self:
        department = self.department
        elected_benefits = self.elected_benefits

        if department == Department.IT and elected_benefits:
            raise ValueError(
                "IT employees are contractors and don't qualify for
   benefits"
            )
        return self
```

ここでは、Pythonの `Self` 型と Pydanticの `model_validator()` をインポートに追加しています。次に、従業員がIT部門に所属し、かつ `elected_benefits` フィールドが `True` である場合にエラーが発生するメソッド `.check_it_benefits()` を作成します。`@model_validator` で `mode` を `"after"` に設定すると、Pydanticはモデルをインスタンス化した後に `.check_it_benefits()` を実行します。

注記: `.check_it_benefits()` にPythonの `Self` 型が型注釈されていることにお気づきかもしれません。これは `.check_it_benefits()` が `Employee` クラスのインスタンスを返すためであり、`Self` 型がこれに対して推奨されるアノテーションだからです。Python 3.11未満のバージョンを使用している場合は、`typing_extensions` から `Self` 型をインポートする必要があります。

新しいモデルバリデータの動作を確認するために、次の例を見てみましょう。

```
>>> from pydantic_models import Employee

>>> new_employee = {
...     "name": "Alexis Tau",
...     "email": "ataue@example.com",
...     "birth_date": "2001-04-12",
...     "compensation": 100_000,
...     "department": "IT",
...     "elected_benefits": True,
... }

>>> Employee.model_validate(new_employee)
Traceback (most recent call last):

pydantic_core._pydantic_core.ValidationError: 1 validation error for
Employee
  Value error, IT employees are contractors and don't qualify for
  benefits.
  [type=value_error, input_value={'name': 'Alexis Tau',
  ...elected_benefits': True},
  input_type=dict]
    For further information visit
    https://errors.pydantic.dev/2.6/v/value_error
```

この例では、`department` を `IT`、`elected_benefits` を `True` に設定して `Employee` モデルを作成しようとしています。`.model_validate()` を呼び出すと、IT部門の従業員は契約社員であるため福利厚生の対象外であることを知らせるエラーをPydanticが発生させます。

モデルバリデータとフィールドバリデータを使用すると、思いつくほぼあらゆるカスタム検証を実装できます。これで、独自のユースケースに合わせてPydanticモデルを作成するための確固たる基礎が得られたはずです。次は趣向を変えて、`BaseModel` のフィールドだけでなく、任意の関数を検証するためにPydanticを使用する方法を見ていきます。

3.2 バリデーションデコレータを使用した関数の検証

`BaseModel` はデータスキーマを検証するためのPydanticの主力クラスですが、`@validate_call` デコレータを使用して関数の引数を検証するためにPydanticを使用することもできます。これにより、検証ロジックを手動で実装することなく、有益な型エラーメッセージを備えた堅牢な関数を作成できます。

これがどのように機能するかを見るために、クライアントが購入を行った後に請求書を送信する関数を作成していると仮定します。この関数は、クライアントの名前、メールアドレス、購入した商品、および請求合計金額を受け取り、メールを作成して送信します。これら

が間違っていると、メールが送信されなかったり、フォーマットが崩れたり、クライアントに誤った請求がなされたりする可能性があるため、これらの入力すべてを検証する必要があります。

これを実現するために、次のような関数を記述します。

validate_functions.py

```
import time
from typing import Annotated
from pydantic import PositiveFloat, Field, EmailStr, validate_call

@validate_call
def send_invoice(
    client_name: Annotated[str, Field(min_length=1)],
    client_email: EmailStr,
    items_purchased: list[str],
    amount_owed: PositiveFloat,
) -> str:

    email_str = f"""
    Dear {client_name}, \n
    Thank you for choosing xyz inc! You
    owe ${amount_owed:,.2f} for the following items: \n
    {items_purchased}
    """

    print(f"Sending email to {client_email}...")
    time.sleep(2)

    return email_str
```

まず、send_invoice() の記述と型注釈に必要な依存関係をインポートします。次に、@validate_call でデコレートされた send_invoice() を作成します。send_invoice() を実行する前に、@validate_call は各入力がアノテーションに適合しているか確認します。この例では、@validate_call は client_name に少なくとも1文字が含まれているか、client_email が正しくフォーマットされているか、items_purchased が文字列のリストであるか、および amount_owed が正の浮動小数点数（float）であるかを確認します。

入力の1つがアノテーションに適合しない場合、Pydanticは BaseModel で見られたものと同様のエラーを発生させます。すべての入力が有効な場合、send_invoice() は文字列を作成し、time.sleep(2) を使ってクライアントへのメール送信をシミュレートします。

注記: client_name がPythonの Annotated 型で注釈されていることにお気づきかもしれません。一般的に、関数の引数に関するメタデータを提供したい場合に Annotated を使用できます。Pydanticは、Field によって指定されたメタデータを持つ関数引数を検証する必要がある場合に Annotated を使用することを推奨しています。

ただし、default_factory を使用して関数の引数にデフォルト値を割り当てる場合は、引数を Field インスタンスに直接割り当てる必要があります。この例はPydanticのドキュメントで確認できます。

@validate_call と send_invoice() の動作を確認するには、新しいPython REPLを開き、次のコードを実行します。

```
>>> from validate_functions import send_invoice

>>> send_invoice(
```

```
...     client_name="",
...     client_email="ajolawsonfakedomain.com",
...     items_purchased=["pie", "cookie", 17],
...     amount_owed=0,
... )
Traceback (most recent call last):
pydantic_core._pydantic_core.ValidationError: 4 validation errors for
send_invoice
client_name
  String should have at least 1 character [type=string_too_short,
  input_value='', input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/string_too_short
client_email
  value is not a valid email address: The email address is not valid.
  It must have exactly one @-sign. [type=value_error,
  input_value='ajolawsonfakedomain.com', input_type=str]
items_purchased.2
  Input should be a valid string [type=string_type, input_value=17,
  input_type=int]
    For further information visit
    https://errors.pydantic.dev/2.6/v/string_type
amount_owed
  Input should be greater than 0 [type=greater_than, input_value=0,
  input_type=int]
    For further information visit
    https://errors.pydantic.dev/2.6/v/greater_than
```

この例では、send_invoice() をインポートし、無効な関数引数を渡しています。Pydanticの @validate_call はこれを認識し、client_name に少なくとも1文字が必要であること、client_email が無効であること、items_purchased に文字列が含まれるべきであること、および amount_owed が0より大きい必要があることを伝えるエラーが発生します。

有効な入力を渡すと、send_invoice() は期待通りに実行されます。

```
>>> email_str = send_invoice(
...     client_name="Andrew Jolawson",
...     client_email="ajolawson@fakedomain.com",
...     items_purchased=["pie", "cookie", "cake"],
...     amount_owed=20,
... )
Sending email to ajolawson@fakedomain.com...

>>> print(email_str)

    Dear Andrew Jolawson,

    Thank you for choosing xyz inc! You
    owe $20.00 for the following items:

    ['pie', 'cookie', 'cake']
```

@validate_call は BaseModel ほど柔軟ではありませんが、関数の引数に強力な検証を適用するために使用できます。これにより、多くの時間を節約でき、ボイラープレートな型チェックや検証ロジックの記述を避けることができます。これまでにこれを行ったことがあるなら、関数の引数ごとに assert 文を書くことがどれほど面倒かご存知でしょう。多くのユースケースにおいて、@validate_call がこれを代わりに処理してくれます。

このチュートリアルの最後のセクションでは、設定管理やコンフィグレーションにPydanticを使用する方法を学びます。

4 設定の管理

Python アプリケーションの設定において最も人気のある方法の 1 つは、環境変数を使用することです。環境変数とは、Python コードの外側（オペレーティングシステム上）に存在する変数であり、自作のコードや他のプログラムから読み取ることができます。環境変数として保存したいデータの例には、シークレットキー、データベースの認証情報、API 認証情報、サーバーアドレス、アクセストークンなどがあります。

環境変数は開発環境と本番環境の間で変更されることが多く、その多くに機密情報が含まれています。このため、コード内で環境変数を解析、検証、統合するための堅牢な方法が必要となります。これは pydantic-settings の最適なユースケースであり、このセクションで詳しく見ていきます。

4.1 BaseSettings によるアプリケーションの設定

pydantic-settings は、Python で環境変数を管理するための最も強力な方法の 1 つであり、FastAPI などの人気ライブラリでも広く採用され、推奨されています。pydantic-settings を使用すると、BaseModel に類似した、環境変数を解析および検証するモデルを作成できます。

pydantic-settings の中心となるクラスは BaseSettings であり、BaseModel とまったく同じ機能を備えています。ただし、BaseSettings を継承したモデルを作成した場合、モデルのイニシャライザはキーワード引数として渡されなかったすべてのフィールドを環境変数から読み取ろうとします。

これがどのように動作するかを見るために、アプリケーションがデータベースと別の API サービスに接続していると仮定します。データベースの認証情報と API キーは時間の経過とともに変更される可能性があり、デプロイ先の環境によって異なることがよくあります。これを処理するために、次のような BaseSettings モデルを作成できます。

settings_management.py

```
from pydantic import HttpUrl, Field
from pydantic_settings import BaseSettings

class AppConfig(BaseSettings):
    database_host: HttpUrl
    database_user: str = Field(min_length=5)
    database_password: str = Field(min_length=10)
    api_key: str = Field(min_length=20)
```

このスクリプトでは、BaseSettings モデルを作成するために必要な依存関係をインポートします。pydantic_settings から BaseSettings をインポートする際、ハイフンではなくアンダースコアを使用していることに注意してください。次に、BaseSettings を継承し、データベースと API キーに関するフィールドを保持する AppConfig モデルを定義します。この例では、database_host は有効な HTTP URL である必要があり、残りのフィールドには最小長の制約があります。

次に、ターミナルを開いて以下の環境変数を追加します。Linux、macOS、または Windows Bash を使用している場合は、export コマンドでこれを行うことができます。

```
(venv) $ export DATABASE_HOST="http://somedatabaseprovider.us-east-2.
    com"
```

```
(venv) $ export DATABASE_USER="username"
(venv) $ export DATABASE_PASSWORD="asdfjl348ghl@9fhsl4"
(venv) $ export API_KEY="ajfsdla48fsdal49fj94jf93-f9dsal"
```

Windows PowerShell で環境変数を設定することもできます。その後、新しい Python REPL を開いて AppConfig をインスタンス化できます。

```
>>> from settings_management import AppConfig

>>> AppConfig()
AppConfig(
    database_host=Url('http://somedatabaseprovider.us-east-2.com/'),
    database_user='username',
    database_password='asdfjl348ghl@9fhsl4',
    api_key='ajfsdla48fsdal49fj94jf93-f9dsal'
)
```

AppConfig をインスタンス化する際にフィールド名を指定していないことに注目してください。代わりに、BaseSettings モデルは設定された環境変数からフィールドを読み取ります。また、環境変数はすべて大文字でエクスポートしましたが、AppConfig はそれらを正常に解析および保存できました。これは、BaseSettings が環境変数をフィールド名と照合する際に大文字と小文字を区別しないためです。

次に、Python REPL を閉じ、無効な環境変数を作成します。

```
(venv) $ export DATABASE_HOST="somedatabaseprovider.us-east-2"
(venv) $ export DATABASE_USER="usee"
(venv) $ export DATABASE_PASSWORD="asdf"
(venv) $ export API_KEY="ajf"
```

別の Python REPL を開き、AppConfig を再度インスタンス化します。

```
>>> from settings_management import AppConfig

>>> AppConfig()
Traceback (most recent call last):
pydantic_core._pydantic_core.ValidationError: 4 validation errors for
AppConfig
database_host
  Input should be a valid URL, relative URL without a base
  [type=url_parsing, input_value='somedatabaseprovider.us-east-2',
  input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/url_parsing
database_user
  String should have at least 5 characters [type=string_too_short,
  input_value='usee', input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/string_too_short
database_password
  String should have at least 10 characters [type=string_too_short,
  input_value='asdf', input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/string_too_short
api_key
  String should have at least 20 characters [type=string_too_short,
  input_value='ajf', input_type=str]
    For further information visit
```

```
    https://errors.pydantic.dev/2.6/v/string_too_short
```

今回、AppConfig をインスタンス化しようとすると、pydantic-settings は database_host が有効な URL ではないこと、および残りのフィールドが最小長の制約を満たしていないことを示すエラーを出力します。

これは単純化された設定例ですが、BaseSettings を活用すれば、環境変数から必要なほぼすべてのデータを解析および検証できます。BaseModel で実行できるすべての検証（モデルバリデータやフィールドバリデータを使用したカスタム検証を含む）は、BaseSettings でも同様に実行できます。

最後に、SettingsConfigDict を使用して BaseSettings の動作をさらにカスタマイズする方法を学びます。

4.2 SettingsConfigDict による設定のカスタマイズ

前の例では、環境変数を解析および検証する BaseSettings モデルの基本的な作成例を見てきました。しかし、BaseSettings モデルの動作をさらにカスタマイズしたい場合もあり、これは SettingsConfigDict を使用して実現できます。

手動で各環境変数をエクスポートすることができず（よくあるケースです）、.env ファイルから読み取る必要がある場合を考えます。解析時に BaseSettings が大文字と小文字を区別するようにし、モデルで指定した変数以外に追加の環境変数が .env ファイルに存在しないようにしたいとします。SettingsConfigDict を使用してこれを行う方法は以下のとおりです。

settings_management.py

```python
from pydantic import HttpUrl, Field
from pydantic_settings import BaseSettings, SettingsConfigDict

class AppConfig(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        case_sensitive=True,
        extra="forbid",
    )

    database_host: HttpUrl
    database_user: str = Field(min_length=5)
    database_password: str = Field(min_length=10)
    api_key: str = Field(min_length=20)
```

このスクリプトは前の例と同じですが、今回は SettingsConfigDict をインポートして AppConfig 内で初期化しています。SettingsConfigDict 内で、環境変数を .env ファイルから読み取ること、大文字と小文字の区別を強制すること、および .env ファイル内の余分な環境変数を禁止することを指定しています。

次に、settings_management.py と同じディレクトリに .env という名前のファイルを作成し、以下の環境変数を記述します。

.env

```
database_host=http://somedatabaseprovider.us-east-2.com/
database_user=username
database_password=asdfjfffff1348ghl@9fhsl4
api_key=ajfsdla48fsdal49fj94jf93-f9dsal
```

これで、Python REPL を開いて AppConfig モデルを初期化できます。

```
>>> from settings_management import AppConfig

>>> AppConfig()
AppConfig(
    database_host=Url('http://somedatabaseprovider.us-east-2.com/'),
    database_user='username',
    database_password='asdfjfffff1348ghl@9fhsl4',
    api_key='ajfsdla48fsdal49fj94jf93-f9dsal'
)
```

ご覧のとおり、AppConfig は .env ファイル内の環境変数の解析と検証に成功しました。

最後に、.env ファイルに無効な変数をいくつか追加してみましょう。

.env

```
DATABASE_HOST=http://somedatabaseprovider.us-east-2.com/
database_user=username
database_password=asdfjfffff1348ghl@9fhsl4
api_key=ajfsdla48fsdal49fj94jf93-f9dsal
extra_var=shouldntbehere
```

ここでは、database_host を DATABASE_HOST に変更して大文字・小文字の区別制約に違反させ、本来あるべきではない余分な環境変数を追加しました。これを検証しようとしたときのモデルの応答は以下のとおりです。

```
>>> from settings_management import AppConfig

>>> AppConfig()
Traceback (most recent call last):
pydantic_core._pydantic_core.ValidationError: 3 validation errors for
AppConfig
database_host
  Field required [type=missing, input_value={'database_user':
  'userna..._var': 'shouldntbehere'}, input_type=dict]
    For further information visit
    https://errors.pydantic.dev/2.6/v/missing
DATABASE_HOST
  Extra inputs are not permitted [type=extra_forbidden,
  input_value='http://somedatabaseprovider.us-east-2.com/', input_type=
   str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/extra_forbidden
extra_var
  Extra inputs are not permitted [type=extra_forbidden,
  input_value='shouldntbehere', input_type=str]
    For further information visit
    https://errors.pydantic.dev/2.6/v/extra_forbidden
```

database_host が不足していること、および .env ファイルに余分な環境変数があることを示す詳細なエラーのリストが表示されます。大文字と小文字の区別制約により、モデルは DATABASE_HOST を extra_var と同様に余分な変数として認識していることに注意してください。

SettingsConfigDict やより一般的な BaseSettings でできることは他にもたくさんありますが、これらの例から、独自のユースケースに合わせて pydantic-settings を使用し環境変数を管理する方法が理解できるはずです。

5 まとめ

Pydantic は、Python における使いやすく、高速で、広く信頼されているデータ検証ライブラリです。Pydantic の概要を広く把握できたので、自身のプロジェクトで Pydantic を使い始めるために必要な知識とリソースが整いました。

このチュートリアルで学んだこと:

- **Pydantic** とは何か、そしてなぜこれほど広く採用されているのか
- Pydantic の**インストール**方法
- `BaseModel` と**バリデータ**を使用してデータスキーマを解析、検証、シリアル化する方法
- `@validate_call` を使用して関数のカスタム検証ロジックを作成する方法
- `pydantic-settings` を使用して環境変数を解析および検証する方法

Pydantic はコードの堅牢性と信頼性を向上させ、Python の使いやすさと静的型付け言語の組み込みデータ検証との間のギャップを部分的に埋めます。データ解析、検証、シリアル化のほぼすべてのユースケースに対して、Pydantic はエレガントなソリューションを提供します。

一度に 1 つのレコードではなくデータテーブル全体を検証する必要がある場合は、Validating Data With Pointblank in Python をご覧ください。